



Powered by OVERNET.

SQL Toolbox For Developers In The Area Of AI

Riccardo Perico

BI Architect / Data Platform MVP

Lucient Italia

Riccardo Perico



riccardo.perico@lucient.it

github.com/R1k91

[@r1k91.bsky.social](https://bsky.social/@r1k91)

[@R1k91](https://x.com/R1k91)

[linkedin.com/in/riccardo-perico](https://www.linkedin.com/in/riccardo-perico)

medium.com/riccardo-perico



BI Architect @ **Lucient**

+15 years in Microsoft “Data Realm”

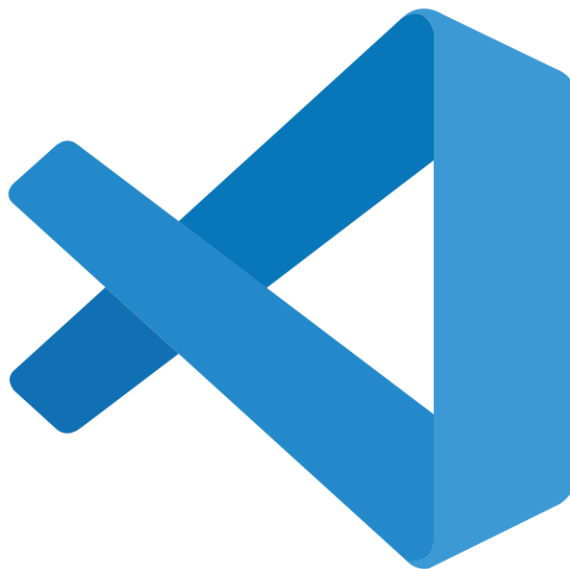
Giving my little contribution to the community





Powered by **OVERNET.**

Where is Microsoft investing?



SSMS is alive and well



2 major releases in 12 months

V21

- Based on new Visual Studio shell
- Leverage Visual Studio installer
- GitHub integration
- Vertical tabs layout
- Export from details
- Dark theme

V22

- AMD64 support in preview
- GitHub Copilot integration
 - GitHub Copilot custom instructions
 - GitHub Copilot database instructions
- New connection dialog
- Details zoom decoupled
- Group objects by schema **FABCON**
- Database projects **FABCON**
- More export formats **FABCON**
(JSON, Markdown, XML and Excel)



Is VSCode really ready?



Azure Data Studio is dead, and now?

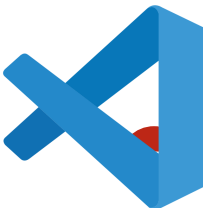
Is Visual Studio Code ready to rock?



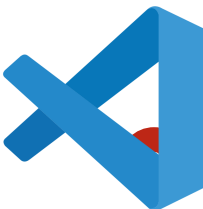
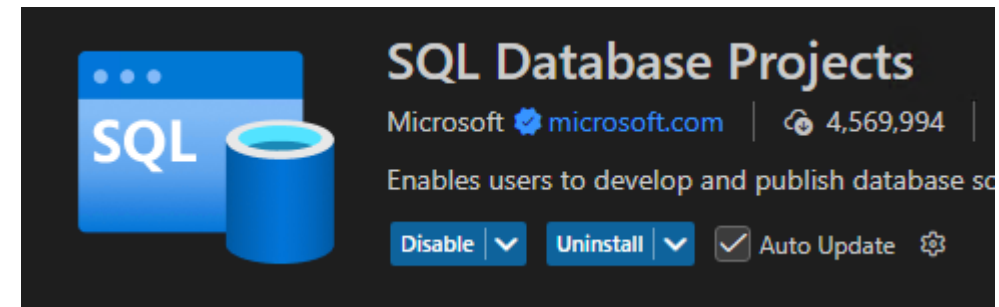
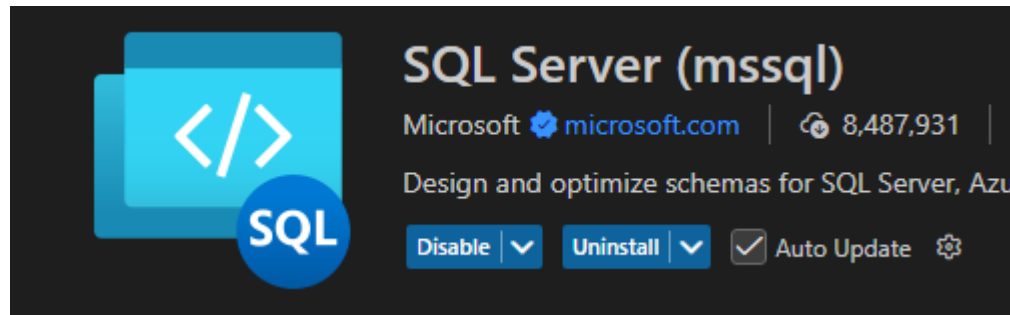
Riccardo Perico · 7 min read · Feb 28, 2025



1



Two extensions needed



Mssql extension



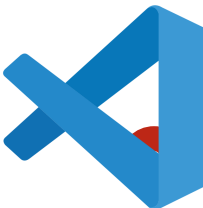
SQL Query Editor showing a query and its results:

```
1 SELECT TOP (1000) [id]
2   , [descr]
3 FROM [MCP].[dbo].[myTable]
```

Query Results (1) Messages

	id	descr
1	1	Sample 1
2	2	Sample 2
3	3	Sample 3
4	4	Sample 4
5	5	Sample 5
6	6	Sample 6
7	7	Sample 7
8	8	Sample 8
9	9	Sample 9
10	10	Sample 10

Switch to text
Save as CSV
Save as JSON
Save as Excel



SQL Notebooks are back



Notebook 1

Ehi! I'm a SQL notebook and I'm back in VScode MSSQL extension

. | AdventureWorks2017

```
select 1 as id
```

✓ 0.0s

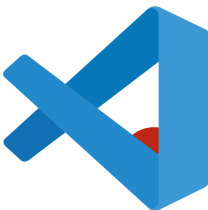
(1 row affected)

	id	↕	⌵
1	1		

Database Projects extension



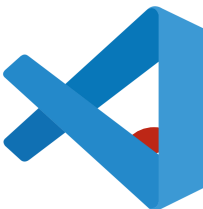
- No offline IntelliSense
- Project explorer and navigation not fluid
- **Buggy schema compare**
- Few templates for create item, even if increasing **FABCON**
- **Missing database settings page**



Database Projects SDK-style



- New database project format
- Make it standard across platforms (SQL Server, Azure SQL, Fabric WH...)
- But... it is preview
- ...super preview



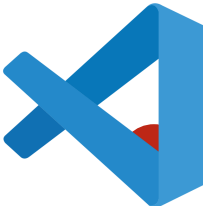
Support in tools is chaotic



	No SDK Style	SDK Style
Visual Studio 2022	✓	✓(*)
Visual Studio 2026	✓	✗
Visual Studio Code	✓	✓
SQL Server Management Studio	✗	✓

```
WideWorldImportersDW.sqlproj X
WideWorldImportersDW > WideWorldImportersDW.sqlproj > Project > PropertyGroup > DefineDebug
2 <Project DefaultTargets="Build" xmlns="http://schemas.microsoft.com/developer/msbuild/2003"
3 <PropertyGroup>
16 <ModelCollation>1033, CI</ModelCollation>
17 <DefaultFileStructure>BySchemaAndSchemaType</DefaultFileStructure>
18 <DeployToDatabase>True</DeployToDatabase>
19 <TargetFrameworkVersion>v4.5</TargetFrameworkVersion>
20 <TargetLanguage>CS</TargetLanguage>
21 <AppDesignerFolder>Properties</AppDesignerFolder>
22 <SqlServerVerification>False</SqlServerVerification>
23 <IncludeCompositeObjects>True</IncludeCompositeObjects>
24 <TargetDatabaseSet>True</TargetDatabaseSet>
25 </PropertyGroup>
26 <PropertyGroup Condition=" '$(Configuration)|$(Platform)' == 'Release|AnyCPU' ">
27 <OutputPath>bin\Release</OutputPath>
28 <BuildScriptName>$(MSBuildProjectName).sql</BuildScriptName>
29 <TreatWarningsAsErrors>False</TreatWarningsAsErrors>
30 <DebugType>pdbonly</DebugType>
31 <Optimize>true</Optimize>
32 <DefineDebug>false</DefineDebug>
33 <DefineTrace>true</DefineTrace>
34 <ErrorReport>prompt</ErrorReport>
35 <WarningLevel>4</WarningLevel>
36 </PropertyGroup>
37 <PropertyGroup Condition=" '$(Configuration)|$(Platform)' == 'Debug|AnyCPU' ">
38 <OutputPath>bin\Debug</OutputPath>
39 <BuildScriptName>$(MSBuildProjectName).sql</BuildScriptName>
40 <TreatWarningsAsErrors>False</TreatWarningsAsErrors>
41 <DebugSymbols>true</DebugSymbols>
42 <DebugType>full</DebugType>
43 <Optimize>false</Optimize>
44 <DefineDebug>true</DefineDebug>
45 <DefineTrace>true</DefineTrace>
46 <ErrorReport>prompt</ErrorReport>
47 <WarningLevel>4</WarningLevel>
48 </PropertyGroup>
49 <PropertyGroup>
50 <VisualStudioVersion Condition=" '$(VisualStudioVersion)' == '' ">11.0</VisualStudioVersion>
51 <!-- Default to the v11.0 targets path if the targets file for the current VS version
52 <SSDTExists Condition="Exists('$(MSBuildExtensionsPath)\Microsoft\VisualStudio\v$(VisualStudioVersion)
53 <VisualStudioVersion Condition=" '$(SSDTExists)' == '' ">11.0</VisualStudioVersion>
54 </PropertyGroup>
55 <Import Condition=" '$(NetCoreBuild)' == 'true' " Project="$(NETCoreTargetsPath)\Microsoft
56 <Import Condition=" '$(NetCoreBuild)' != 'true' AND '$(SQLDBExtensionsRefPath)' != '' " Project="$(SQLDBExtensionsRefPath)
57 <Import Condition=" '$(NetCoreBuild)' != 'true' AND '$(SQLDBExtensionsRefPath)' == '' " Project="$(SQLDBExtensionsRefPath)
58 <ItemGroup>
59 <PackageReference Condition=" '$(NetCoreBuild)' == 'true' " Include="Microsoft.NETFramework
60 </ItemGroup>
61 <ItemGroup>
62 <Folder Include="Properties" />
63 </ItemGroup>
64 <ItemGroup>
65 <None Include=".vscode\tasks.json" />
66 </ItemGroup>
67 <ItemGroup>
68 <Build Include="Application\StoredProcedures\Configuration_ApplyPartitionedColumnstore
69 <Build Include="Application\StoredProcedures\Configuration_ApplyPolybase.sql" />
70 <Build Include="Application\StoredProcedures\Configuration_ConfigureForEnterpriseEdition
71 <Build Include="Application\StoredProcedures\Configuration_EnableInMemory.sql" />
72 <Build Include="Application\StoredProcedures\Configuration_PopulateLargeSaleTable.sql" />
73 <Build Include="Application\StoredProcedures\Configuration_ReseedETL.sql" />
74 <Build Include="Dimension\Tables\City.sql" />
```

```
WWISDK.sqlproj X
WWISDK > WWISDK.sqlproj > Project
1 <?xml version="1.0" encoding="utf-8"?>
2 <Project DefaultTargets="Build">
3 <Sdk Name="Microsoft.Build.Sql" Version="1.0.0" />
4 <PropertyGroup>
5 <Name>WWISDK</Name>
6 <ProjectGuid>{55A4A879-0A35-40CB-A407-766FE9DBF5F6}</ProjectGuid>
7 <DSP>Microsoft.Data.Tools.Schema.Sql.Sql160DatabaseSchemaProvider</DSP>
8 <ModelCollation>1033, CI</ModelCollation>
9 </PropertyGroup>
10 <ItemGroup>
11 <None Include=".vscode\tasks.json" />
12 </ItemGroup>
13 <Target Name="BeforeBuild">
14 <Delete Files="$(BaseIntermediateOutputPath)\project.assets.json" />
15 </Target>
16 </Project>
```



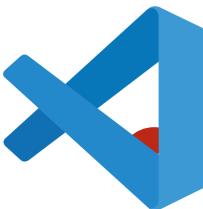
Feature	VS Code	SSMS	SSDT (VS2022-2026)	SDK-style SSDT, preview (VS2022)
Create new empty project	Yes	Yes	Yes	Yes
Create new project from existing database	Yes	Yes	Yes	Yes
Open existing Microsoft.Build.Sql projects	Yes	Yes	No	Yes
Open original-style (SSDT) projects	Yes	No	Yes	No
Solution management and operations	No	Yes	Yes	Yes
Project run build	Yes	Yes	Yes	Yes
Publish project to existing server	Yes	Yes	Yes	Yes
Publish project to a local development instance	Yes ¹	Yes ²	Yes ³	Yes ³
Publish options/properties	Yes	No	Yes	Yes



The reference matrix



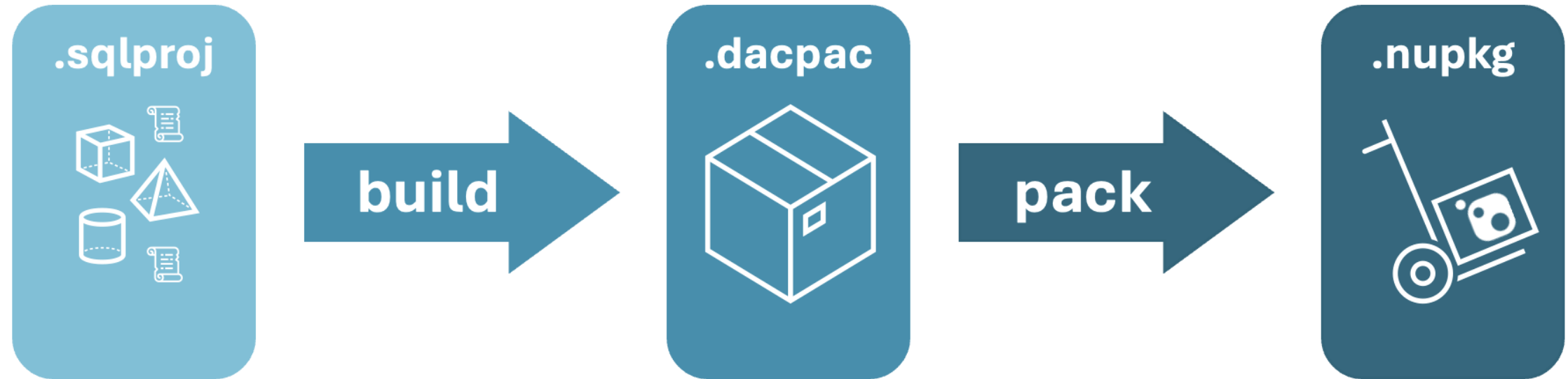
		Old Style	SDK Style
Old Style	project ref	✓	✗
Old Style	dacpac ref	✓	✗
SDK Style	project ref	✗	Not Recomend
SDK Style	dacpac ref	✓	Not Recomend
SDK Style	package (nuget ref)	✗	✓



It could be the future if you need

- .NET 8+ support (cross platform)
- NuGet [package references](#) for database references
- Default globbing pattern for .sql files in the project

Database package reference (NuGet)



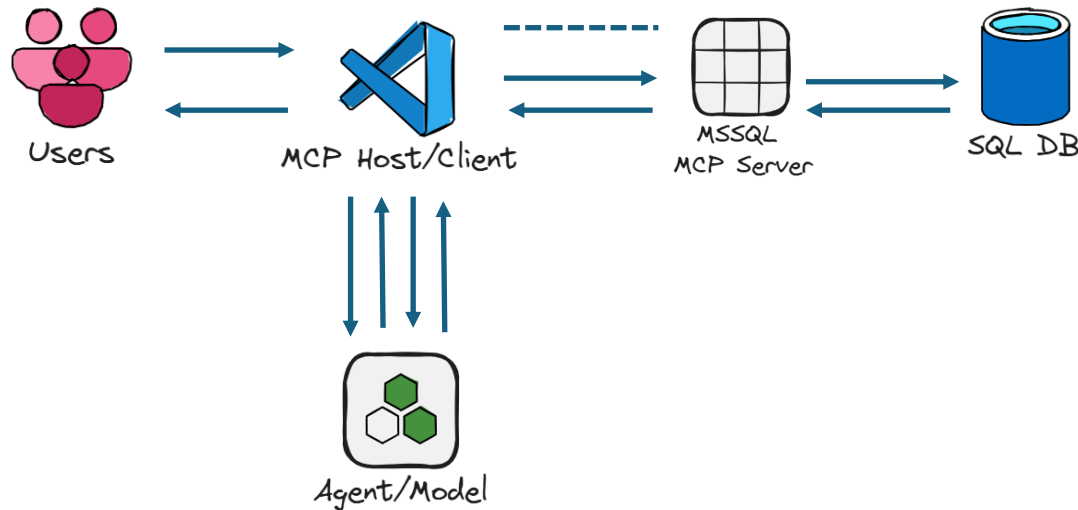


GitHub Copilot

Real world success cases?

- Built T-SQL code based on pattern
- Scanned 82K lines of code to migrate/optimize
 - Documenting process
 - Defining object dependencies
- Defined job processes
 - Got calls stack in job steps
 - For each stored list of views and tables referenced
 - For each view list of tables referenced
 - For each table referenced list of fields referenced

MSSQL MCP Server



1. User chat the request to MCP Host: “Give me one customer”
2. MCP Host connects to MCP Server
3. MCP Host discovers Tools: “get_data”
4. MCP Host sends request to Agent: “I have these tools and user wants a customer”
5. Agent interprets the request and suggest how to call the tool: “use get_data tool doing SELECT TOP(1) FROM Cust”
6. MCP Host call the tool using JSON-RPC
7. MCP Server connect to Server and execute the query
8. Server return a response to MCP Server
9. MCP Server sends result in JSON-RPC to MCP Host
10. MCP Host sends result to Agent to format
11. Agent format the result in natural language and send back to MCP Host
12. MCP Host render the result





Demo

Useful links

- [Azure Data Studio Retirement Announcement](#)
- [Azure Data Studio is dead, and now?](#)
- [SQL Server Management Studio \(SSMS\) 21 Changes](#)
- [Limitations SQL Server Data Tools, SDK-Style \(Preview\)](#)
- [Feature comparison SQL Projects Tools](#)
- [GitHub MssqlMcp repo](#)
- [Introducing MSSQL MCP Server \(Preview\)](#)



Q&A

Riccardo Perico



riccardo.perico@lucient.it

github.com/R1k91

[@r1k91.bsky.social](https://bsky.social/@r1k91)

[@R1k91](https://x.com/R1k91)

[linkedin.com/in/riccardo-perico](https://www.linkedin.com/in/riccardo-perico)

medium.com/riccardo-perico



BI Architect @ **Lucient**

+15 years in Microsoft “Data Realm”

Giving my little contribution to the community





Powered by OVERNET.

Special Thanks to our Event Host



acer